

● EASY STREET • MEDELLÍN SOCIAL

Platform & GoHighLevel Architecture Blueprint

How the custom Medellín Social platform and GoHighLevel work together as one system with two specialised layers — not two competing databases.

PREPARED FOR

Edward — Application & Data Architecture
Talal — GoHighLevel Architecture
Ken Monroe — Founder

SCOPE

Data ownership, sync engine, CRM layer,
provisioning, community, global scaling

STATUS

Technical planning draft
for review & validation

One platform. Two specialised layers.

The core decision this document settles: GoHighLevel is **not** the application database. It is the business operations layer that wraps around it.

APPLICATION DATA LAYER

Website / FastAPI / PostgreSQL + PostGIS

Owns every listing, property, business record, user account, geographic boundary and spatial query. This is the source of truth and the engine behind the public site.

BUSINESS OPERATIONS LAYER

GoHighLevel

Owns the customer relationship: CRM, payments, subscriptions, onboarding, automation, community, affiliates, calendars and each paying client's own sub-account.

1 PostgreSQL stays master. PostGIS spatial logic is not rebuilt inside GHL.

2 FastAPI is the bridge. All synchronisation runs through the API, not the browser.

3 One-way sync first. Website → GHL at launch; two-way only where it earns its cost.

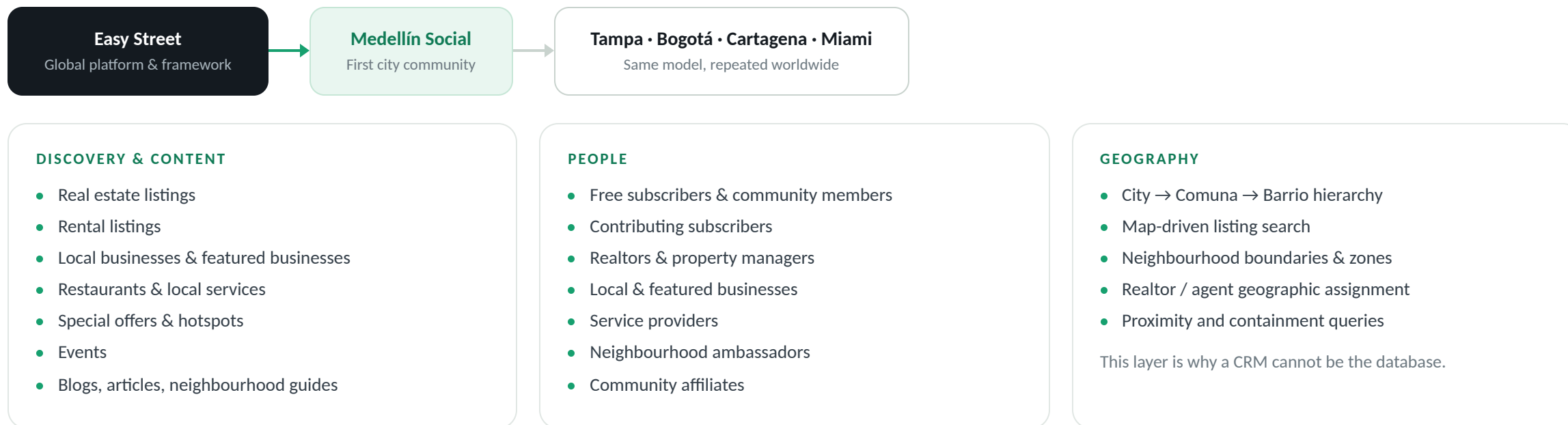
4 Multi-city from day one. Every record carries a community ID. Nothing is hardcoded to Medellín.

The failure mode we are designing out: a platform where listings live as custom fields on GHL contacts, spatial search is impossible, and the public website goes dark whenever GHL has an outage.

CONTEXT

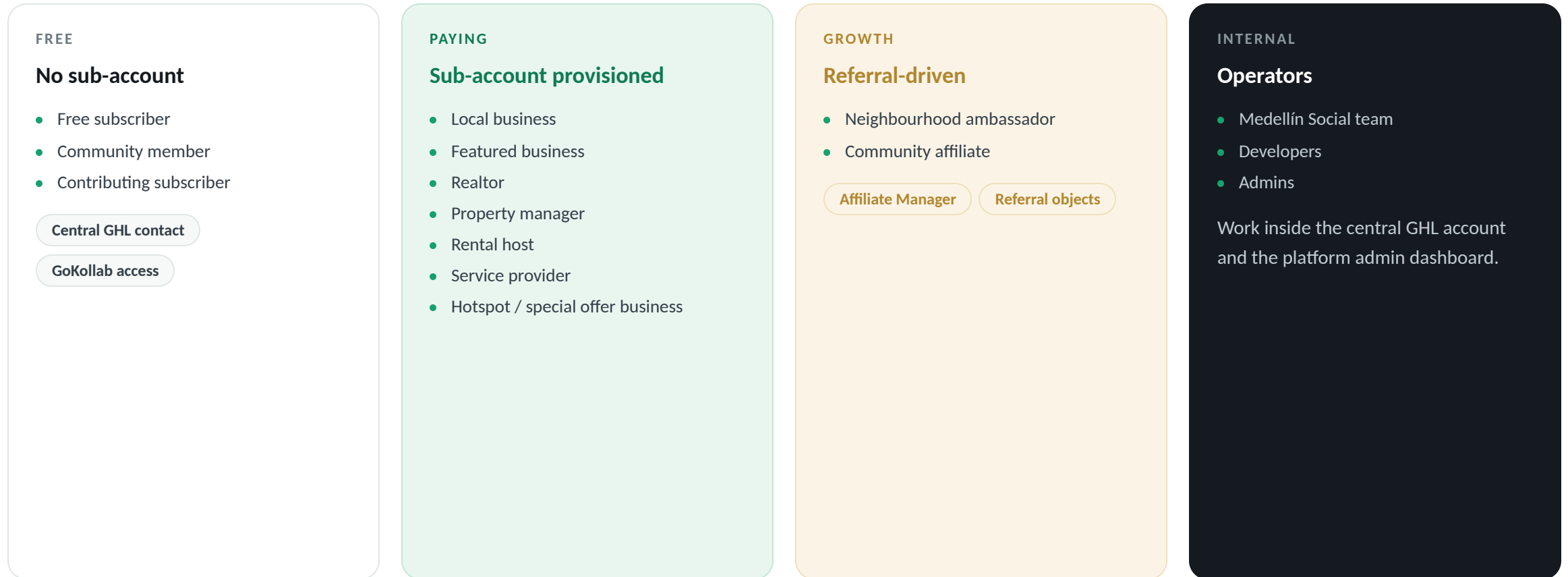
What Medellín Social is

A virtual local community — a modern digital Chamber of Commerce. Easy Street is the parent framework; Medellín is the first city implementation of many.



User & customer types

Customer type drives everything downstream: which snapshot is applied, whether a sub-account is created, and which custom objects sync to them.



Design implication: customer_type must be a first-class field in PostgreSQL *and* on the central GHL contact. Provisioning automation branches entirely on this value.

STARTING POINT

The stack Edward has already built

This architecture adds to what exists. Nothing here is being replaced or migrated away.

FRONTEND

- TanStack Start
- React / TypeScript

BACKEND

- FastAPI
- Python

HOSTING

- Railway (app + API)
- Railway cron workers

PAYMENTS

- GoHighLevel

DATABASE — POSTGRESQL + POSTGIS

public

staging

analytics

raw

public holds live transactional data: listings, agents, agencies, users, visit requests and all PostGIS geography.

SPATIAL LOGIC IN ACTIVE USE

ST_Contains

ST_DWithin

neighbourhood queries

zone assignment

agent geo-assignment

map-area listing search

None of this has an equivalent inside a CRM. It stays in PostGIS.

MEDIA & OBJECT STORAGE — CLOUDFLARE R2

Property photos, business imagery, documents, raw scraped data, Parquet / Iceberg analytics datasets.

SCHEDULED PROCESSING

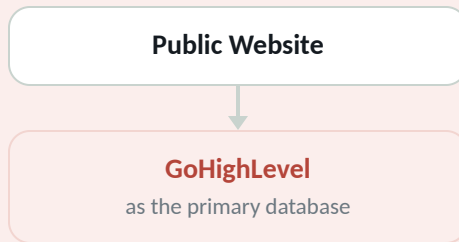
Railway cron workers today. Airflow currently runs from a developer machine and should move to persistent cloud / VPS infrastructure.

Do not build the website on top of GHL

The single most important decision in this document. Everything that follows depends on it.

× ANTI-PATTERN

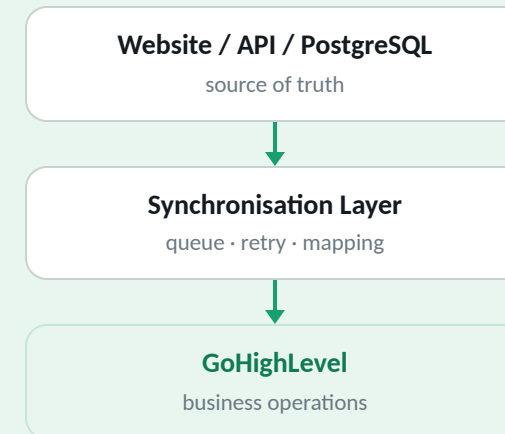
Website → GHL as database



- No spatial queries — map search breaks
- Listings become repeated contact fields
- Site goes down whenever GHL does
- API rate limits become page-load limits
- Impossible to scale to many cities

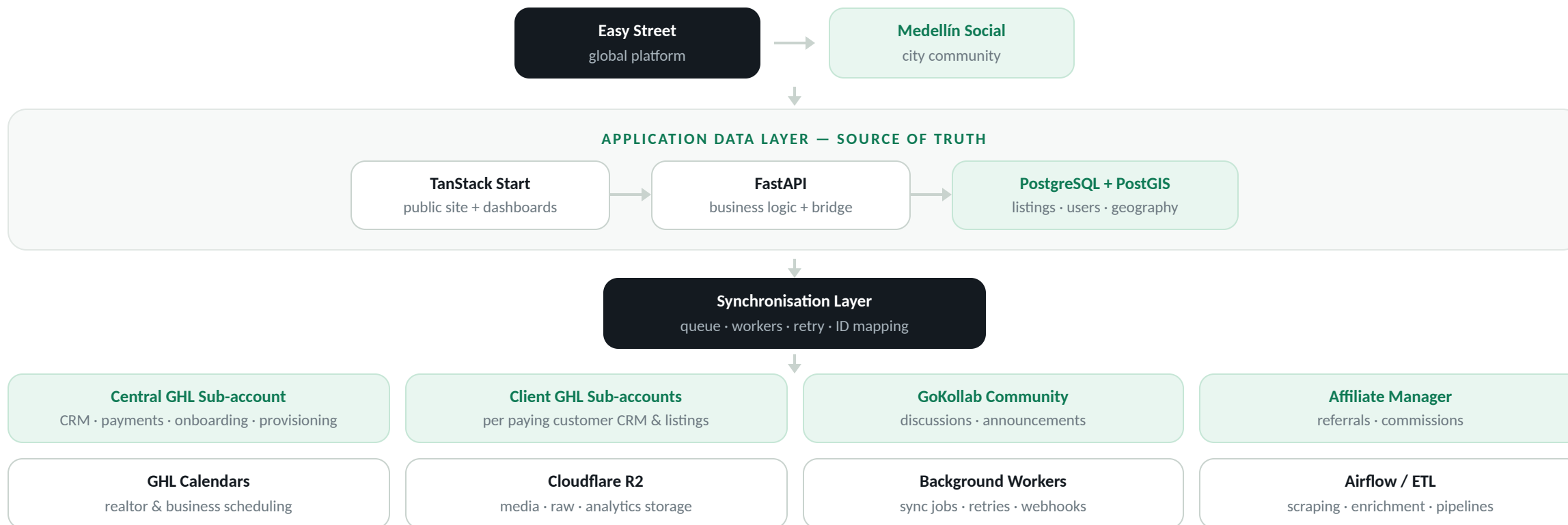
✓ CORRECT PATTERN

Website → Sync Layer → GHL



- PostGIS keeps doing what it is good at
- GHL keeps doing what it is good at
- Website survives GHL outages
- Sync failures are retried, not fatal
- Same model repeats for every new city

High-level platform architecture



What each platform owns

If a question ever arises about where something belongs, this slide answers it.

WEBSITE • FASTAPI • POSTGRESQL + POSTGIS

The application & public platform

- Property & rental listings
- Business directory records
- Events, offers, articles, blogs
- Neighbourhood & barrio data
- All PostGIS geometry & boundaries
- Map & proximity search
- User accounts & site sessions
- Visit requests (visita_solicitud)
- Media references in R2
- Analytics & reporting datasets

Every public page renders from PostgreSQL. No page-load ever depends on a GHL API call.

GOHIGHLEVEL

The business operations layer

- Contacts & customer records
- Order forms, payments, subscriptions
- Onboarding & lifecycle workflows
- Email, SMS, voice, conversations
- Sales pipelines & opportunities
- Client sub-account provisioning
- Snapshots & templates
- Affiliate Manager & commissions
- GoKollab community access
- Calendars & appointments

GHL holds a **synchronised, business-facing copy** of listings and businesses — never the master copy.

Source of truth, record by record

For every entity there is exactly one system that owns it. The other system holds a copy and is allowed to be briefly stale.

ENTITY	MASTER	SYNCHRONISED COPY	WHY
Property listing	PostgreSQL	GHL custom object	Spatial search, map rendering, public SEO pages
Rental listing	PostgreSQL	GHL custom object	Same query and geography requirements as property
Business record	PostgreSQL	GHL custom object	Directory search, category filtering, barrio filtering
Neighbourhood / geography	PostGIS	— not synced —	No CRM equivalent exists for polygons and containment
Person / customer	GoHighLevel	Reference row in PostgreSQL	CRM, marketing consent, conversations, lifecycle
Payment / subscription	GoHighLevel	Status mirrored to platform	GHL processes payments and holds the billing record
Affiliate commission	GHL Affiliate Manager	Referral custom object	Financial attribution must have one authority
Appointment	GHL Calendar	visita_solicitud + GHL ID	Realtor works the calendar; platform keeps the analytics copy
Media & images	Cloudflare R2	URLs referenced in both	Object storage, served via CDN

Rule of thumb: if it appears on a public page or in a map query, PostgreSQL owns it. If it describes a relationship with a human being or money, GHL owns it.

The central Medellín Social GHL account

One master sub-account runs the community's own business. It is the hub every customer passes through.

CRM & RECORDS

- Subscriber records
- Paying customer records
- Customer type tracking
- Package tracking
- Internal pipelines

ACQUISITION

- Signup forms
- Order forms
- Payments
- Subscriptions
- Main sales pipeline

ENGAGEMENT

- Workflow automation
- Email · SMS · voice
- Conversations
- Onboarding sequences
- Community access grants

GROWTH & PROVISIONING

- Affiliate Manager
- Referral attribution
- GoKollab
- Client sub-account creation
- Snapshot assignment

EXPLICITLY OUT OF SCOPE

The central GHL account is **not** the primary real-estate or listing database. It must never become the place where a listing is first created or where the authoritative price, address or geography lives.

Client GHL sub-accounts

Paying customers receive their own CRM environment — a genuine product benefit, provisioned automatically at purchase.

WHO RECEIVES ONE

- Realtor
- Local business
- Service provider
- Property manager
- Rental business
- Restaurant
- Salon
- Med spa
- Professional services

WHAT LANDS INSIDE THEIR ACCOUNT

- Their own contacts & leads
- Their listings as custom objects
- Their referrals
- Their pipeline
- Their calendars
- Their email & SMS
- Their conversations
- Their marketing automations

AUTOMATED PROVISIONING SEQUENCE

- 1 • Create contact
- 2 • Detect customer type
- 3 • Create sub-account
- 4 • Apply snapshot
- 5 • Create user
- 6 • Send login details
- 7 • Install workflows
- 8 • Install custom objects
- 9 • Configure CRM
- 10 • Configure pipeline
- 11 • Configure calendar
- 12 • Grant community access
- 13 • Start onboarding — client becomes active

Proposed — validate sub-account creation, snapshot assignment and user creation against current GHL API capabilities and agency plan limits.

COMPARISON

Central GHL vs client GHL

Two very different jobs. Keeping them separate prevents customer data leaking into the community's own CRM and vice versa.

ONE ACCOUNT FOR THE WHOLE COMMUNITY

Central Medellín Social GHL

- Main CRM
- Membership & subscriptions
- Payments
- Main signup & order forms
- Affiliate Manager
- GoKollab community
- Onboarding
- Client provisioning
- Master customer lifecycle
- Internal workflows
- Main sales pipeline
- Snapshot library

Audience: the Medellín Social team.

ONE ACCOUNT PER PAYING CUSTOMER

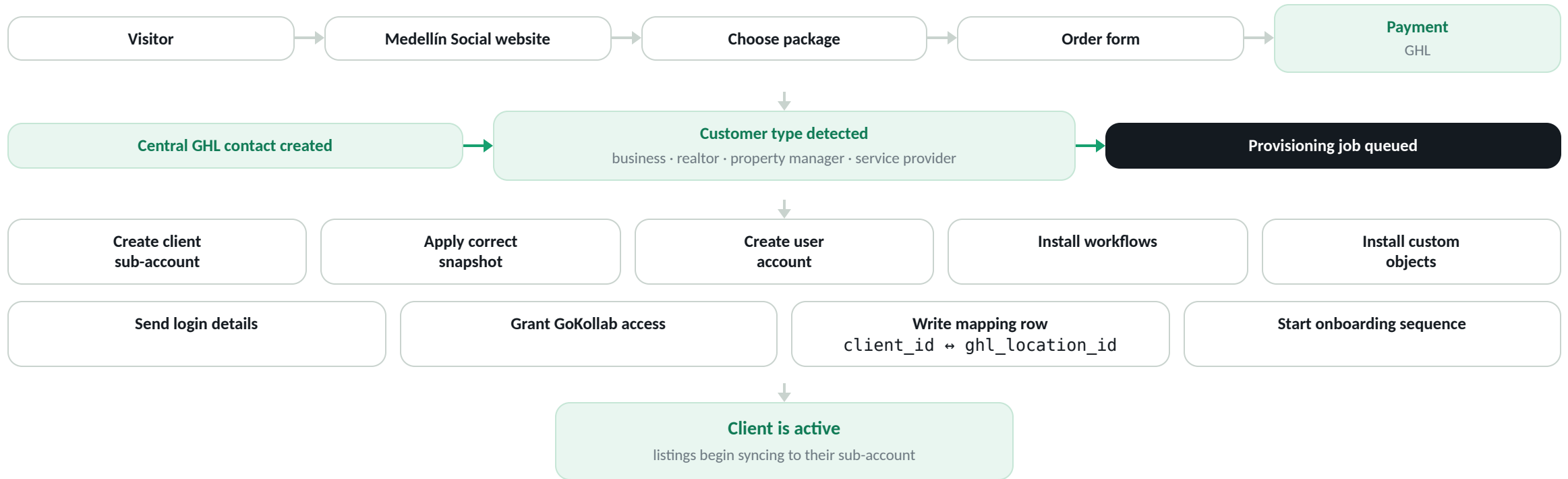
Client GHL sub-account

- Their own CRM
- Their contacts
- Their leads
- Their listings
- Their referrals
- Their pipeline
- Their calendars
- Their email & SMS
- Their conversations
- Their marketing
- Their automations
- Their business operations

Audience: the realtor, business owner or service provider.

The bridge between them is the mapping table: `client_id` → `ghl_location_id`. Without it the platform cannot tell which sub-account a listing belongs in.

Purchase & automated provisioning

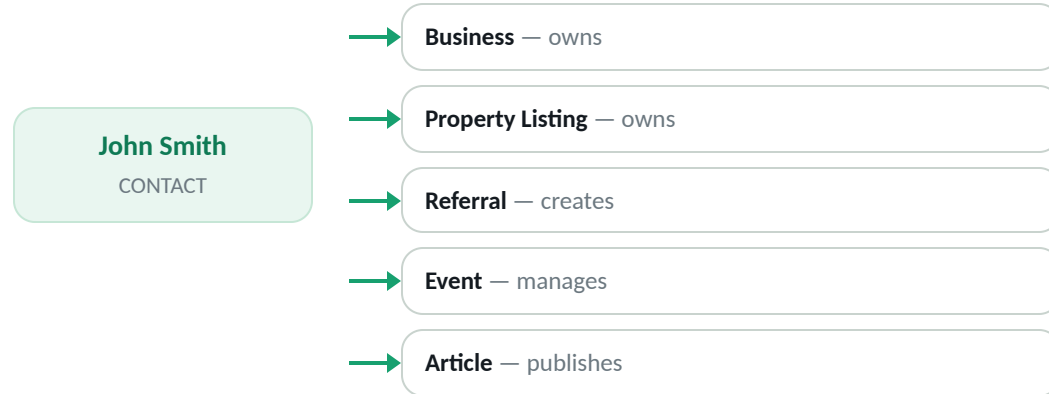


Proposed — validate automated sub-account creation and snapshot deployment against current GHL API capabilities.

Contact = WHO • Custom Object = WHAT

A GHL contact represents a person. Everything that person owns, creates or manages is a separate related record.

THE RELATIONSHIP MODEL



CONTACT — WHO

A single human being. Name, email, phone, consent, conversations, lifecycle stage.

CUSTOM OBJECT — WHAT

A thing that person owns or created. Many per contact, each with its own fields and status.

The same person can be a subscriber, a business owner and an affiliate at once — one contact, three related object types.

× NEVER DO THIS

Flattening many listings into custom fields on one contact:

```
contact.property_1_address  
contact.property_1_price  
contact.property_2_address  
contact.property_2_price  
contact.property_3_address ...
```

Unqueryable, capped at whatever number of fields you guessed, and impossible to report on.

✓ DO THIS INSTEAD



Each listing is its own record, related to the contact. Unlimited, filterable, reportable.

Proposed custom objects

Seven objects cover the platform. Each is created and owned in PostgreSQL, then synchronised into the relevant GHL account.

CUSTOM OBJECT	REPRESENTS	LIVES IN	SYNCED FROM WEBSITE	RELATED TO
Business	A listed local or featured business	Owner's sub-account + central	Yes	Contact (owner)
Property Listing	A for-sale real estate listing	Realtor's sub-account	Yes	Contact (realtor)
Rental Listing	A rental / short-stay property	Host or manager's sub-account	Yes	Contact (host)
Realtor Profile	Professional profile & coverage area	Central + realtor's sub-account	Yes	Contact (realtor)
Referral	An affiliate-facing referral record	Affiliate's sub-account	Generated at conversion	Contact (affiliate)
Event	A community or business event	Organiser's sub-account + central	Yes	Contact · Business
Special Offer	A subscriber deal or hotspot promotion	Business owner's sub-account	Yes	Business

Every synced object stores the returned GHL object ID back in PostgreSQL. That ID is what makes the second sync an **update** instead of a duplicate.

Scope discipline: start with Business, Property Listing and Referral. Add the rest once the sync engine is proven in production.

Use native features first, custom objects second

Anything GHL already does natively should not be rebuilt as a custom object. Custom objects are only for entities GHL has no concept of.

NATIVE GHL FEATURES — USE AS-IS

Contacts	People, users, leads, customers
Opportunities	Sales pipeline & lead progression
Calendars	Appointments & availability
Affiliate Manager	Attribution, commissions, referral links
GoKollab	Community & member interaction
Payments	Orders & subscriptions
Workflows	Automation & triggers
Conversations	SMS, email, chat
Snapshots	Client account provisioning

CUSTOM OBJECTS — ONLY FOR THESE

- Business listings
- Property listings
- Rental listings
- Referrals
- Events
- Special offers
- Realtor profiles

These are Easy Street domain concepts. GHL has no native equivalent, so they become custom objects related to the contact who owns them.

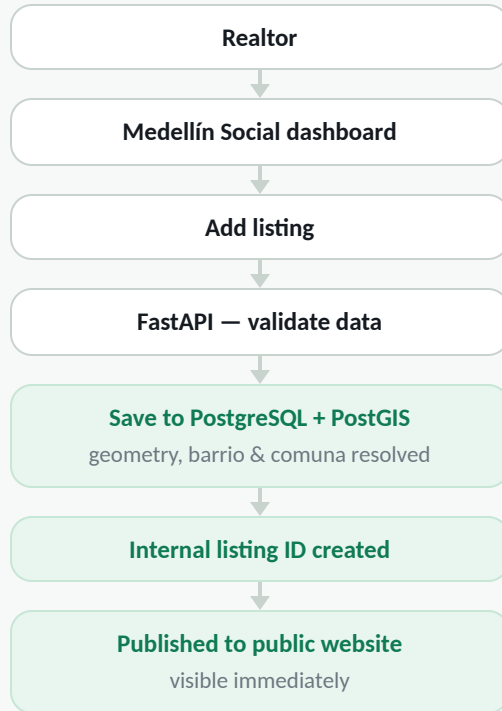
THE TEST

Before creating a custom object, ask: “Does GHL already model this?” If yes, use the native feature. Duplicating native functionality creates two places to look and two places to break.

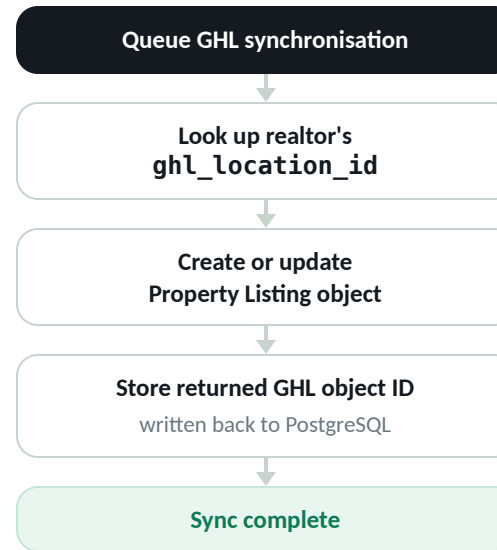
Real estate listing flow

PostgreSQL is the master. GHL receives a synchronised, business-facing copy after the listing is already live.

WEBSITE PATH — SYNCHRONOUS



GHL PATH — ASYNCHRONOUS



If this path fails, the listing is still live. The job simply retries.

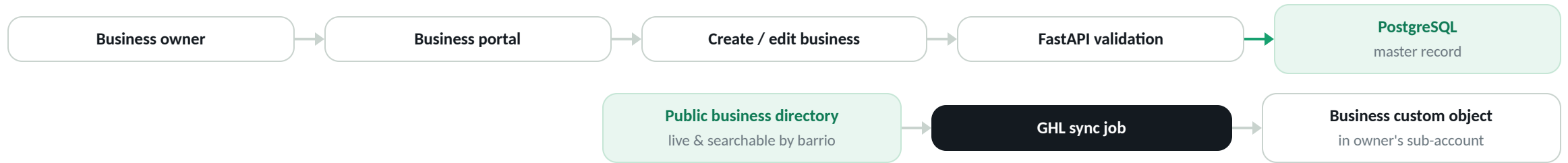
FIELDS CARRIED TO GHL

- Property ID (internal)
- Listing title
- Address
- Barrio · Comuna
- Coordinates
- Price
- Bedrooms · Bathrooms
- Listing type
- Realtor
- Media URLs (R2)
- Status
- Created / updated dates

FLOW

Business listing flow

Identical shape to the real estate flow. One sync engine serves every object type.



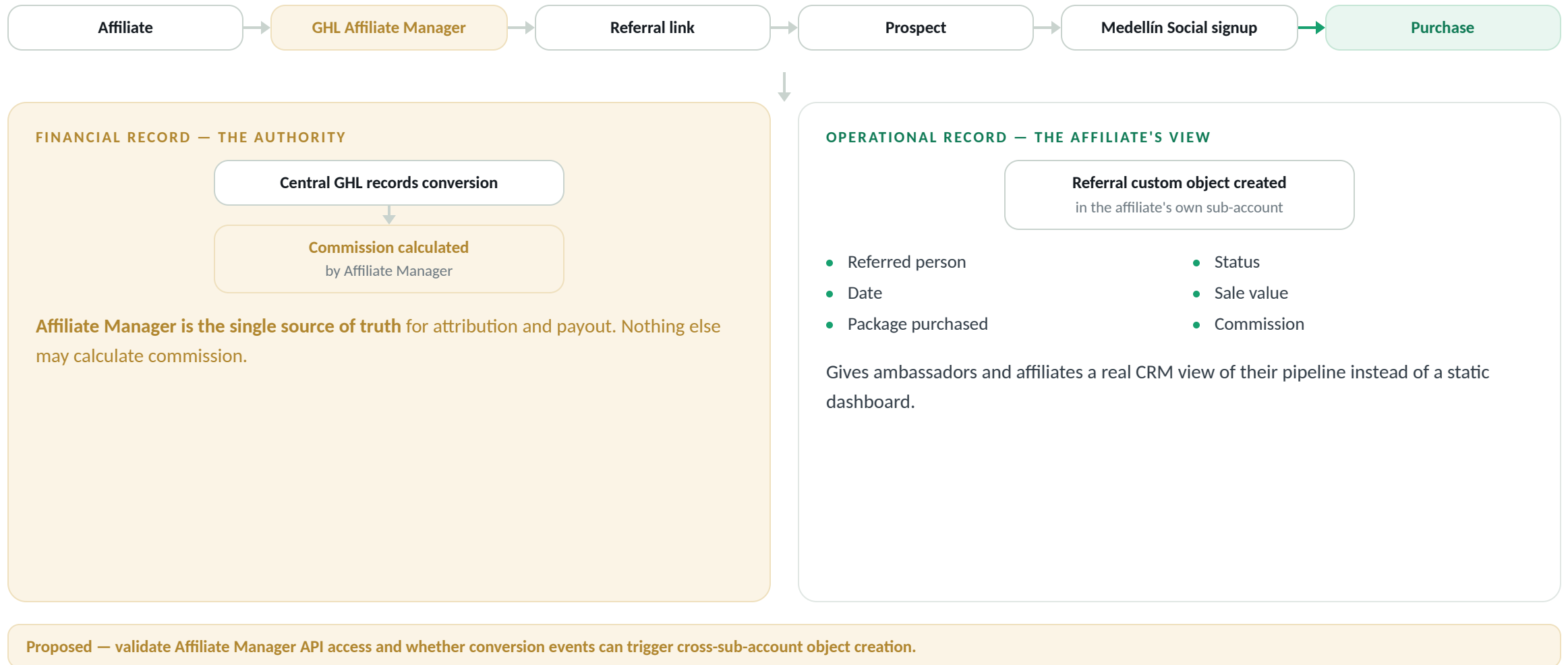
BUSINESS RECORD FIELDS

- | | | | |
|------------------|----------------|---------------------------|-----------------------|
| • Business name | • Email | • Address | • Logo |
| • Category | • Phone | • Barrio | • Banner |
| • Tagline | • WhatsApp | • Comuna | • Gallery |
| • Description | • Website | • Google Business Profile | • Subscriber discount |
| • Business hours | • Social links | • Coordinates (PostGIS) | • Special offers |

Images live in **Cloudflare R2**. Both PostgreSQL and GHM store URLs — neither stores the binary.

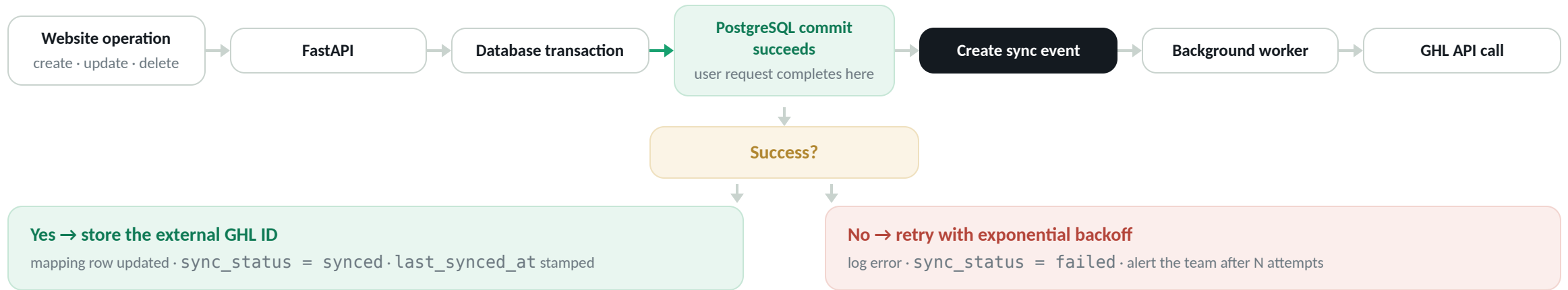
Affiliate & referral flow

Two records with two different jobs: one is the financial authority, the other is the affiliate's working view.



Website → GHL sync engine

The technical heart of the integration. Sync happens *after* the database commit, never inside the user's request.



Both outcomes are written to the object mapping table, so the team can always answer “is GHL up to date?” without comparing records by hand.

NON-NEGOTIABLE

A user-facing action must never fail because GHL is slow or unavailable. The listing saves, the page publishes, and the sync catches up when GHL returns.

IDEMPOTENT

Safe to replay — a retry never creates a duplicate.

ORDERED PER RECORD

A stale job can never overwrite a newer value.

OBSERVABLE

Every failure is logged, alertable and re-runnable.

Mapping tables & external IDs

These two tables are what prevent duplicate records and let the API know exactly which GHL record belongs to which website record.

CLIENT → GHL ACCOUNT MAPPING

```
client_id
community_id
ghl_location_id
ghl_user_id
customer_type
snapshot_id
```

Answers: “Which GHL sub-account does this customer own, and which snapshot were they provisioned with?”

- Written during provisioning
- Read on every object sync
- `community_id` keeps it multi-city from day one

OBJECT → GHL RECORD MAPPING

```
internal_object_id
object_type
ghl_location_id
ghl_object_id
sync_status
last_synced_at · updated_at
sync_error
```

Answers: “Has this listing already been pushed, and where did it land?”

- Missing `ghl_object_id` → create
- Present → update
- `sync_status` drives the retry queue and an operations dashboard

Without these tables every sync run risks creating a second copy of the same listing, and no one can answer “is GHL up to date?” without manually comparing records.

One-way now. Two-way only if it earns it.

PHASE 1 — RECOMMENDED FOR LAUNCH

Website → GHL

Build this

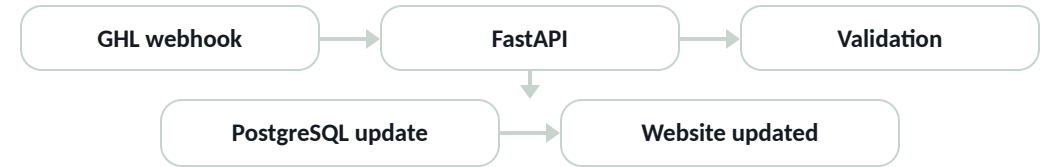


- Simpler to build and reason about
- Safer — one writer per record
- Website remains unambiguous source of truth
- Far easier to debug
- No conflict resolution needed

PHASE 2 — LATER, SELECTIVELY

Website ↔ GHL

Defer



THE PROBLEM TO SOLVE FIRST

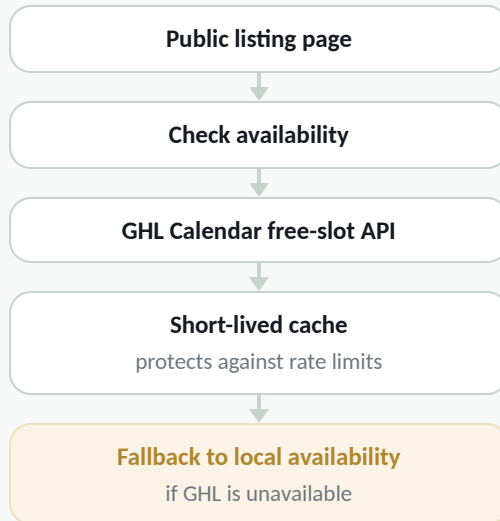


Which value wins? Two-way sync cannot ship until per-field conflict-resolution rules and timestamps are agreed.

Realtor calendar & booking flow

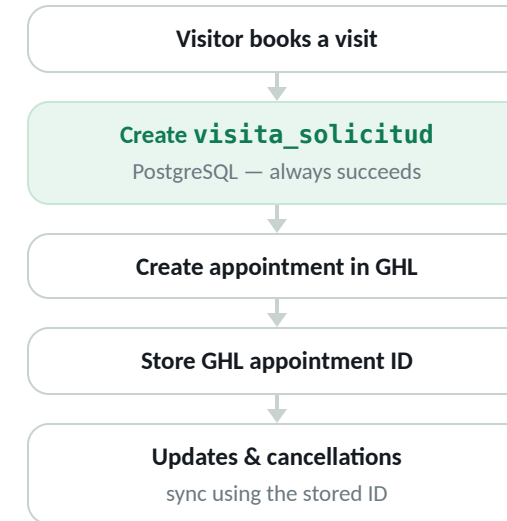
GHL becomes the realtor-facing scheduling interface; PostgreSQL keeps the analytics and source-of-truth copy of every visit request.

AVAILABILITY — READ PATH



The realtor manages their day in a familiar GHL calendar, with reminders and automations already attached.

BOOKING — WRITE PATH



The platform keeps every visit request for reporting, conversion analysis and lead quality — independent of GHL.

GoKollab is the community, not the website

Two distinct surfaces with two distinct jobs. Confusing them leads to duplicated content and a fragmented member experience.

PUBLIC WEBSITE — DISCOVERY

Anyone, indexed by search engines

- Search
- Property listings
- Rentals
- Businesses
- Events
- Special offers
- Blogs & articles
- Neighbourhood guides
- Maps
- Realtor profiles

GOKOLLAB — INTERACTION

Members only, conversation-driven

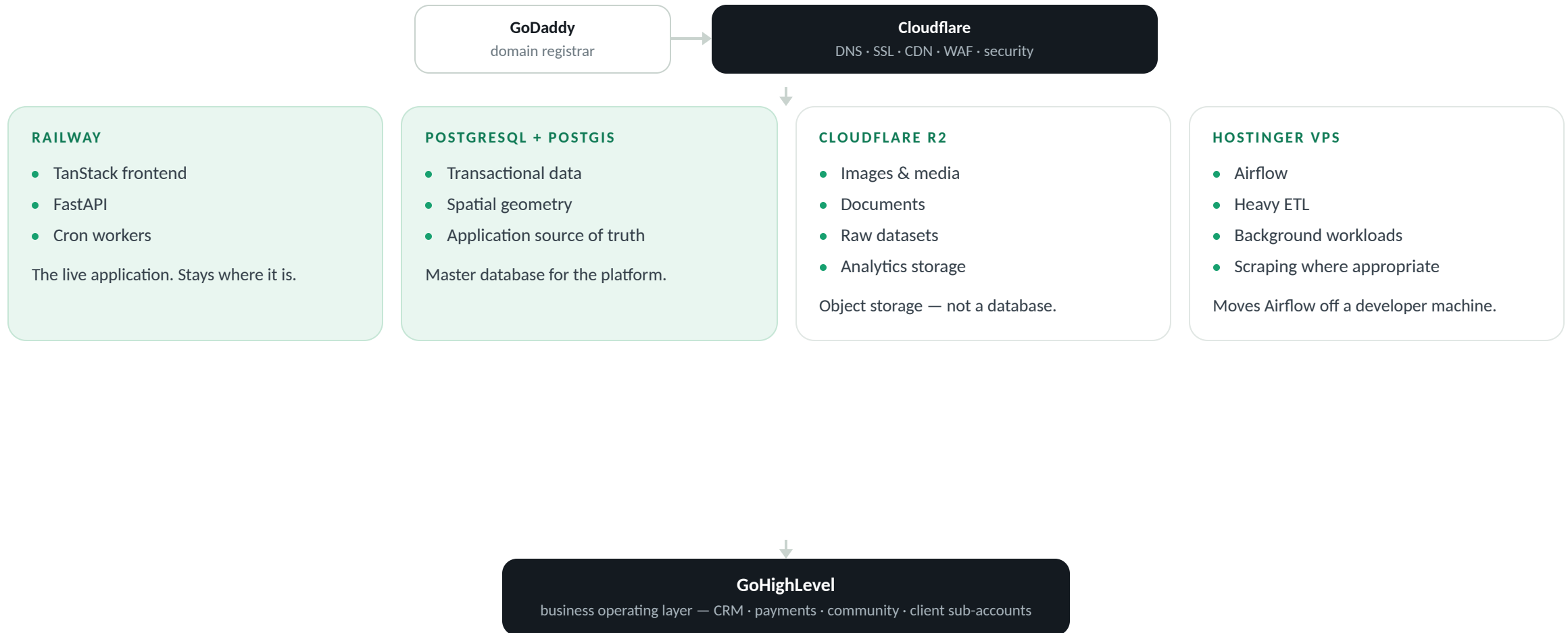
- Discussions
- Community posts
- Announcements
- Learning content
- Member interaction
- Community events
- Channels
- Direct messaging

CURRENT / RECOMMENDED CHANNELS

- Start Here
- Official Announcements
- General Community
- Neighborhood Updates
- Special Offers
- Events
- Real Estate
- Rentals
- Featured Business Members

Community access is granted by GHL workflow at signup or purchase — it is a benefit of membership, tied to the same contact record.

Infrastructure architecture



Hostinger VPS absorbs heavy background jobs **without** migrating the live application away from Railway. Two different workloads, two appropriate homes.

PostgreSQL vs R2 — what moves, what stays

R2 is cheap, durable object storage. It is not a replacement for a transactional database. Move schemas selectively, by access pattern.

STAYS IN POSTGRESQL

public

- Transactional, live writes
- PostGIS geometry & queries
- Every user-facing request

staging

- Spatial joins
- Rebuilt frequently
- Needs PostGIS to function

CANDIDATES TO MOVE LATER

raw

- Large and append-heavy
- Rarely queried after processing
- Good fit for Parquet + DuckDB

analytics

- Read-heavy, batch-written
- Possible R2 Data Catalog / SQL
- No latency requirement

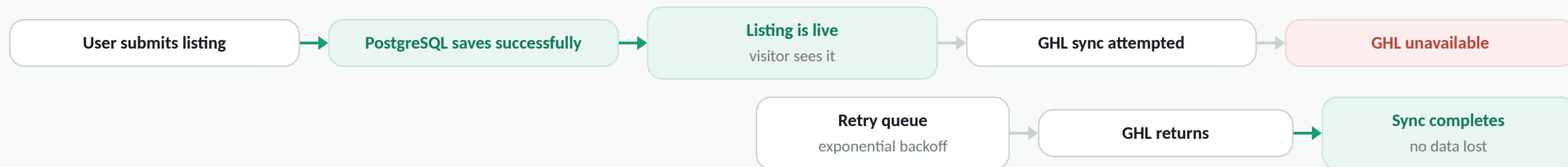
DO NOT

Do not move everything to R2. Anything that serves a live page, powers a spatial query or accepts a user write stays in PostgreSQL. R2 is for cold, bulk and binary data.

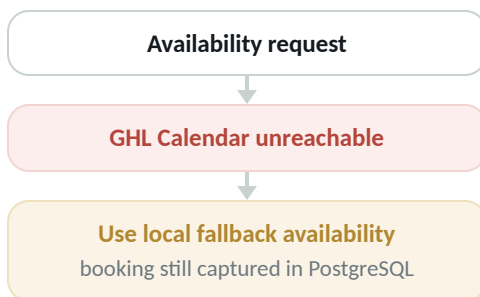
Failure & resiliency design

GHL downtime must be an inconvenience for the operations team, never an outage for the public platform.

SCENARIO 1 — GHL IS UNAVAILABLE DURING A LISTING SUBMISSION



SCENARIO 2 — GHL CALENDAR UNAVAILABLE

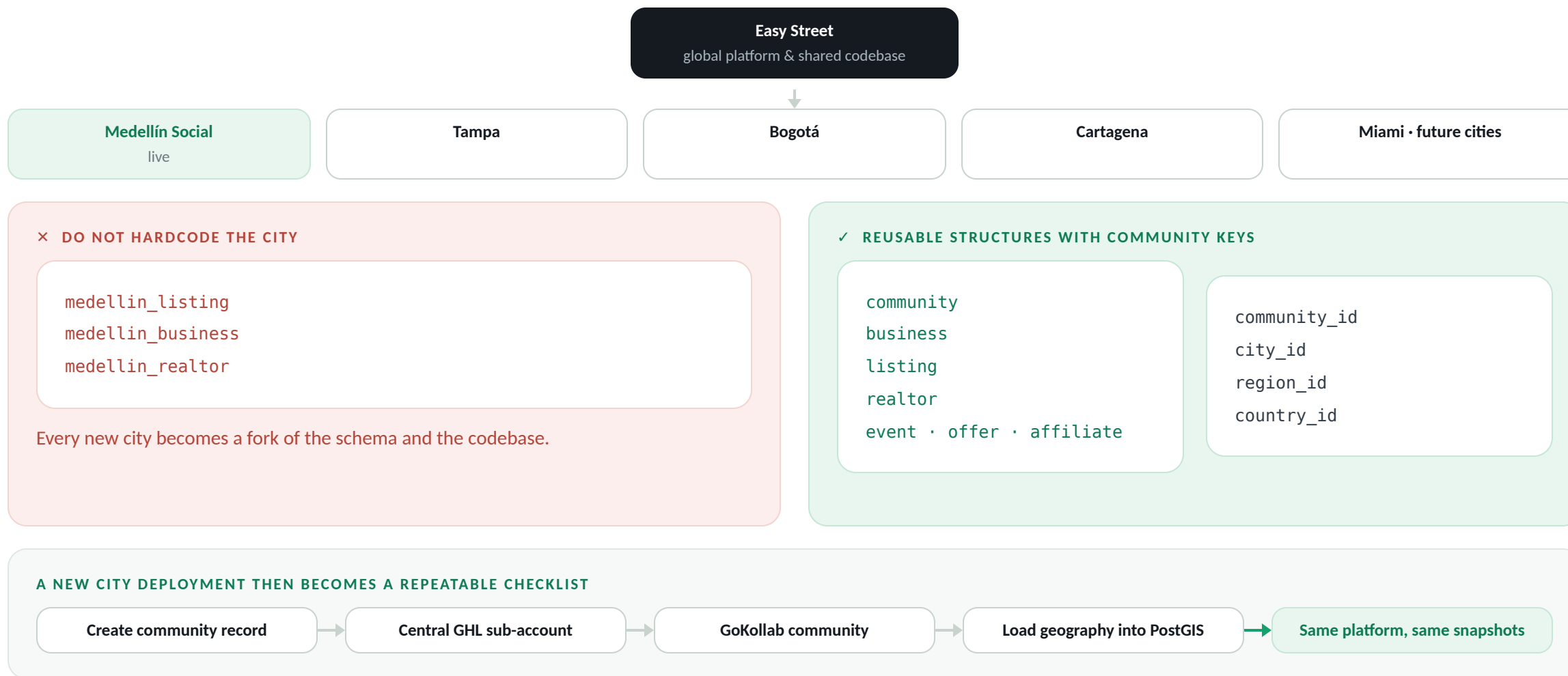


WHAT THIS BUYS THE BUSINESS

- Public site uptime is decoupled from a third-party SaaS
- No lost leads during a GHL incident
- Sync backlog is visible and drainable, not silent data loss
- Rate limits become a queue-depth problem, not a user-facing error
- Failed syncs are alertable and re-runnable by the team

Built for many cities from day one

Medellín is the first implementation, not the shape of the schema. Decisions made now determine whether city two takes a week or a rewrite.



Implementation roadmap

Seven phases. Each one is useful on its own and de-risks the next.

PHASE 1

Architecture foundation

- Confirm data ownership
- Confirm GHL agency structure
- Define custom objects
- Define API mappings
- Define customer types
- Define snapshots
- Define community IDs
- Add monitoring & logging

PHASE 2

Central GHL

- Main CRM
- Signup forms
- Order forms
- Payments
- GoKollab
- Affiliate Manager
- Onboarding
- Workflows

PHASE 3

Client provisioning

- Automatic sub-account creation
- Snapshot deployment
- User provisioning
- Client login delivery
- Client CRM setup

PHASE 4

Website → GHL sync

- Business
- Realtor profile
- Property listing
- Rental listing
- Referral
- Event
- Special offer

PHASE 5

Calendar

- GHL free-slot API
- Appointment synchronisation
- Local fallback availability

PHASE 6

Advanced sync

- GHL webhooks
- Optional GHL → website updates
- Conflict resolution rules

PHASE 7

Global scaling

- Additional Easy Street communities
- Repeatable city deployment
- Standardised GHL templates & snapshots

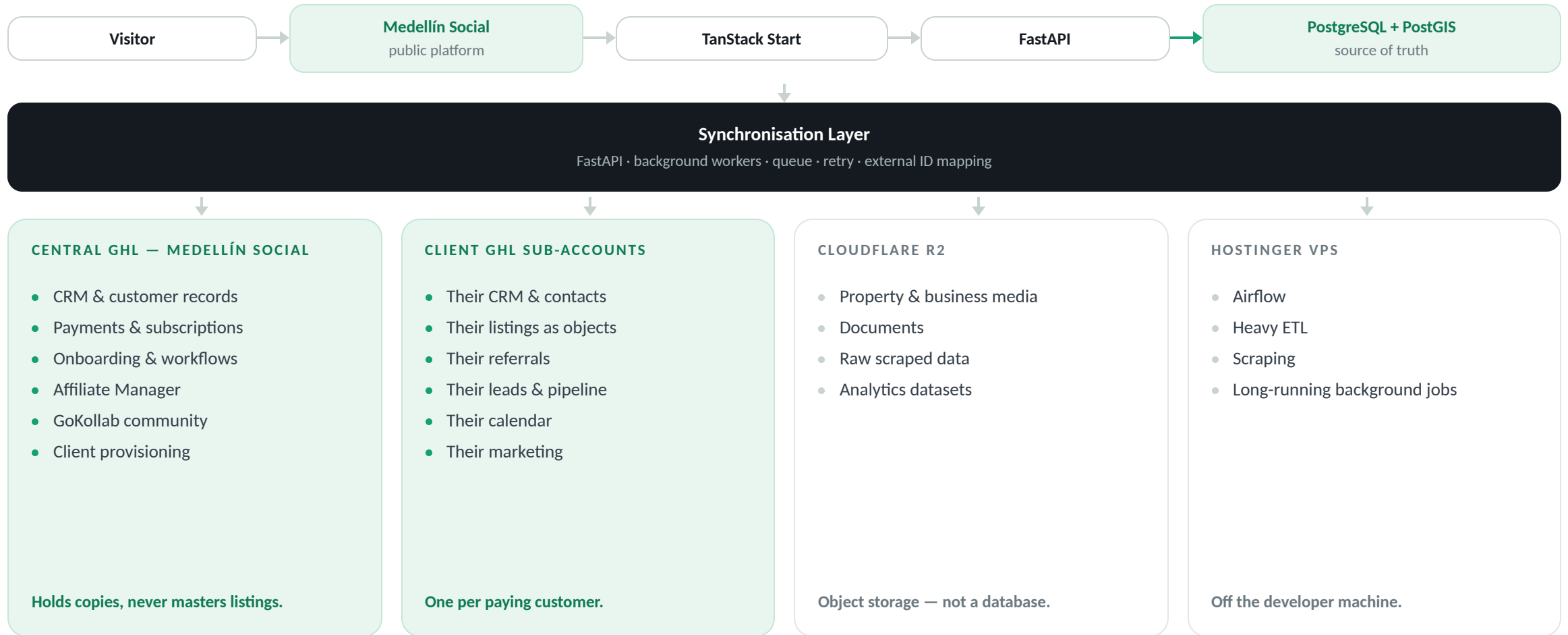
Rules Edward should build around

Twelve constraints that keep the architecture coherent as the platform grows.

- | | | | |
|---|---|----|---|
| 1 | PostgreSQL / PostGIS remains the main application database. | 7 | Listings are separate records, never repeated contact fields. |
| 2 | Do not rebuild spatial or listing logic inside GHL. | 8 | Use background synchronisation with queues and retries. |
| 3 | GHL is an integrated business operations layer — not a second database. | 9 | The website must survive GHL outages. |
| 4 | FastAPI is the integration bridge. No browser-to-GHL writes. | 10 | Start with one-way sync — website to GHL. |
| 5 | Use internal IDs and store GHL external IDs alongside them. | 11 | Add two-way sync only where business value requires it, with conflict rules agreed first. |
| 6 | Every client sub-account needs a location mapping row. | 12 | Design for many cities from day one. |

Any proposal that breaks one of these rules should be treated as an architecture decision requiring explicit sign-off — not an implementation detail.

Master architecture



Read left to right, then down. Data is created and owned on the top row. Everything below the synchronisation layer holds a copy, a service or a workload — never the master record.

IN ONE PARAGRAPH

Medellín Social should not be built as a website connected to a second database called GHL. It should be built as one platform with two specialised layers.

LAYER ONE

PostgreSQL + PostGIS

Manages the application and public platform data — listings, businesses, geography, search.

LAYER TWO

GoHighLevel

Manages business relationships — CRM, automation, community, affiliates, calendars, payments, client operations.

THE CONNECTION

FastAPI

Connects the two — validated, queued, retried, and mapped by external ID.